Environment management for scientific computing

A computer used for scientific analysis must be set up for the work, much as a bench is set up for an experiment. An environment manager creates an isolated, fully specified set of software dependencies for each project that can be shared with collaborators or restored years or more later without manual archaeology.

The one-time install takes about ten minutes; creating an environment for a new project adds roughly two minutes. This is the full overhead. Skipping it works until an analysis needs to be revisited after a software update, at which point reconstructing a working environment from scratch typically takes longer than the setup would have.

This protocol uses Conda, a fast, lightweight environment manager widely used in scientific computing, installed via Miniforge using Mamba as the solver. It handles Python and R packages as well as most command-line tools used in bioinformatics and data science in a unified manner.

Risk assessment

- Installing software from unverified sources can introduce malware or compromise data integrity.
- Mixing system-wide and user-level installs can leave the system in a state that is hard to recover.

▷ Install package managers and packages only from reputable sources: official conda-forge and bioconda channels, the official miniforge installer, your operating system's vendor repositories.

▷ Avoid running unfamiliar shell installers piped from the internet without inspecting them first. Conda and mamba operate at the user level and should not require `sudo` (superuser privileges); if a command asks for it, treat that as a signal to stop and check.



Reviewed: May 10, 2026

Procedures

›› *One-time install on a new machine*

(1.) Download the miniforge installer for the OS and CPU architecture from Miniforge, then run it.

```
bash Miniforge3-$(uname)-$(uname -m).sh
```

Resource: Miniforge (https://github.com/conda-forge/miniforge) is a Conda edition that uses free and openly-licensed packages from the conda-forge project by default which avoids accidental violations of the Anaconda terms of service. It takes up significantly less space and installs much faster than Anaconda.

Hint: Pick `Miniforge3-MacOSX-arm64.sh` for Apple Silicon, `…-x86_64.sh` for Intel Mac or Linux. Accept the license, install into the default `~/miniforge3` path.

Critical: Do not run the installer with `sudo`. Conda installs under your user account and never needs administrator rights.

(2.) Open a new terminal. Verify the install and set the channel configuration.

```
mamba init
mamba info
mamba config --add channels bioconda
mamba config --add channels conda-forge
mamba config --set channel_priority strict
mamba install -n base conda-lock
```

This is why: Strict priority prevents the solver from mixing packages across channels (a common source of broken environments). `--add` prepends to the channel list, so the last call wins; adding conda-forge second places it above bioconda.

This is why: `conda-lock` is a tool to generate a file that records the exact state of a Conda environment ("lockfile"). After installation in the `base` environment, it is available to all projects without cluttering individual environments.

›› *Per-project environment workflow*

(1.) Create one environment per project, named after the repository. Create it from an existing environment specification file `environment.yml`, or from scratch. Specify the Python (and R) version explicitly to avoid silent upgrades when conda-forge advances.

```
mamba env create -n my-project -f environment.yml
mamba activate my-project
```

If no environment file exists yet:

```
mamba create -n my-project python=3.12
mamba activate my-project
```

> *Critical:* Keep the `base` environment empty except for mamba itself. Installing project-related packages into `base` is the most common path to a fragile installation.

> *This is why:* Mamba stores each package once in a shared cache (`~/miniforge3/pkgs/`) and hardlinks it into every environment that uses it. So, ten environments using "numpy" occupy the same disk space as one. The exception is packages installed via `pip`, which copy rather than hardlink. Prefer the conda-forge or bioconda version when one exists.

> *Hint:* To create the environment in a project-local directory instead of the default `~/miniforge3/envs/` for projects that ship their own environment, use `--prefix ./env` and activate with `mamba activate ./env`.

(2.) Install all packages the project needs (Python libraries, R packages, and command-line tools), then export and lock the environment immediately.

```
mamba install numpy pandas scipy matplotlib samtools bwa multiqc
mamba env export --from-history > environment.yml
conda-lock lock -f environment.yml
```

> *Critical:* Install command-line bioinformatics tools (`samtools`, `bwa`, `bcftools`, `nextflow`, etc.) via mamba from bioconda, not through Homebrew or apt. System-installed tools are invisible to `environment.yml` and will not travel with the project.

> *This is why:* The `environment.yml` expresses intent ("this project needs numpy and pandas"); the lockfile expresses fact ("on this date the resolution was these exact versions"). Commit both to the project repository.

> *This is why:* `--from-history` exports only the packages you explicitly requested, producing a portable file that re-solves on another machine. Without it, the export pins the entire transitive graph including platform-specific build hashes.

(3.) When a new package is needed mid-project, install it, then re-export and re-lock as above.

Finding packages

(1.) Search for a package by name from the command line:

```
mamba search numpy
mamba search r-ggplot2
mamba search samtools
```

> *This is why:* Package names on conda-forge follow predictable conventions: R packages use the prefix `r-` followed by the lowercase CRAN name (`r-mass`, `r-glmnet`, `r-parsnip`); Bioconductor packages use `bioconductor-` (`bioconductor-deseq2`); Python and CLI packages usually match their common name. You can often predict the conda name before searching.

> *Hint:* To restrict the search to a specific channel, add `-c conda-forge` or `-c bioconda`. To search across both: `mamba search -c conda-forge -c bioconda samtools`.

(2.) Alternatively, browse the channel web interfaces.

> *Resource:* Use the conda-forge package index (https://conda-forge.org/packages/) or the bioconda package index (https://bioconda.github.io/conda-package_index.html) which focuses on bioinformatics tools and Bioconductor packages. The Pixi package index (https://prefix.dev/) offers a unified search engine across conda-forge, bioconda, and other channels in one interface.

›› *Sharing and managing environment files*

(1.) When working with Git to track changes SOP 0101, commit both `environment.yml` and the lockfile to the project repository alongside the source code. When one changes, regenerate and commit the other in the same commit.

> *Critical:* Updating `environment.yml` without regenerating the lockfile leaves the two artifacts inconsistent. The next collaborator who installs from the lockfile will not see your changes to the environment file; the next who installs from `environment.yml` will get a different version set than you tested.

(2.) Pin top-level packages with major and minor versions in `environment.yml`; leave patches floating.

> *Hint:* Specify `numpy=1.26` rather than `numpy` alone (too loose) or a fully-specified build hash (too tight). The lockfile captures the patch version separately for full reproducibility.

(3.) To recreate an environment exactly from a lockfile on a new machine, install from the lockfile rather than the environment file.

```
conda-lock install --name my-project conda-lock.yml
```

(4.) When working with an external repository, apply the same discipline. If forking to modify or extend: adopt the upstream `environment.yml` if one exists (create one if not), then immediately regenerate a lockfile in the fork.

```
mamba env create -f environment.yml
mamba activate my-project
```

Trial upgrade of a pinned dependency

(1.) Before changing anything, capture the current working state so the original environment can be rebuilt from scratch if both it and the trial are lost.

```
mamba env export -n my-project --from-history > environment.yml
conda-lock lock -f environment.yml
```

> *Critical:* Do not skip this step even if `environment.yml` and the lockfile look up to date. The lockfile is the actual safety net; everything that follows can be discarded as long as it remains valid.

> *Hint:* This procedure covers upgrading a pinned dependency — typically the interpreter (Python or R) or a runtime library other code depends on. Adding a new package mid-project is the simpler case already covered above.

(2.) Create a Git branch SOP 0101 so the YAML edit and any code changes the upgrade forces (deprecation fixes, syntax updates, dependency adjustments) stay isolated from the working `main` branch.

```
git checkout -b try-r45
```

(3.) Edit `environment.yml` on the branch. Bump the pin to try, and loosen any other pins likely to block the upgrade (typically transitive numerical or ML libraries with strict interpreter-ABI requirements).

```
# environment.yml - diff
- - r-base=4.4
+ - r-base=4.5
```

Regenerate the lockfile from the edited YAML. This is the feasibility test: if the solver cannot satisfy the new pin set, the failure surfaces here, before building any environment.

```
conda-lock lock -f environment.yml
```

> *This is why:* A fresh solve from the edited YAML avoids ABI half-states. Running `mamba install python=3.14` on a clone built against py3.12 replaces the interpreter but may leave dependents as cached old-ABI wheels. The env runs, `mamba list` looks fine, but imports fail mysteriously. The lockfile produced from a fresh solve is internally consistent by construction; it crosses interpreter minor versions cleanly where clone+install cannot.

(4.) Build the trial environment from the new lockfile in a single solve, under a new name.

```
conda-lock install --name my-project-r45 conda-lock.yml
mamba activate my-project-r45
```

Run the project's analyses against the trial env and confirm the results match the original. Commit any required code fixes to the `try-r45` branch along the way.

> ***Hint:*** Activate the original environment in a second terminal to compare runs side-by-side without switching back and forth.

(5.) If the upgrade works, promote the trial: retire the original, rename the trial, and merge the branch.

```
mamba env remove -n my-project
mamba rename -n my-project-r45 my-project
git checkout main
git merge try-r45
```

The updated `environment.yml` and lockfile are already on the branch from the YAML-edit step, so the merge brings the spec across together with any code changes.

(6.) If the upgrade breaks, discard the trial environment and the branch. The original environment, `environment.yml`, lockfile, and `main` branch are untouched.

```
mamba env remove -n my-project-r45
git checkout main
git branch -D try-r45
```

> ***Hint:*** Record what you tried (target version, what failed) in the project README or an issue, so the next attempt does not repeat the same combination blindly.

Recovering a broken environment

(1.) When an env misbehaves (failed install, cache corruption, "nothing to do" when some packages are clearly missing, mysterious post-transaction states, accidental `pip install` etc.), delete and rebuild from the lockfile rather than debugging in place.

```
mamba deactivate
mamba env remove -n my-project
conda-lock install --name my-project conda-lock.yml
mamba activate my-project
```

If this succeeds, recovery is complete: the env is bit-identical to the committed state.

> ***This is why:*** Conda environments are derived artefacts; the lockfile is the source of truth. The rebuild works regardless of what went wrong. No diagnosis required.

(2.) If no committed lockfile exists, or installing from it fails (channels evicted a pinned build, the host platform changed, etc.), regenerate a new lockfile from `environment.yml` first:

```
conda-lock lock -f environment.yml
conda-lock install --name my-project conda-lock.yml
mamba activate my-project
```

Commit the regenerated lockfile in the same operation. The project now has a working lockfile again.

> ***Critical:*** This path re-runs the solver against the current channel state. The resulting env may resolve to different transitive versions than the one that was working — especially after a Python or R minor version has advanced. If the regenerated lockfile fails to solve at all, the YAML's pins are jointly infeasible against current channels; loosen the offending pins (typically transitive numerical or ML libraries with strict interpreter-ABI requirements) deliberately, then re-lock.

Updating and pruning

(1.) Update mamba periodically. Update only in `base`, not in project environments.

```
mamba update -n base mamba
```

> ***Hint:*** Project environments should change only when the project's `environment.yml` and lockfile change. Updating mamba in a project environment risks a silent dependency shift.

(2.) List existing environments and remove ones for projects no longer worked on.

```
mamba env list
mamba env remove -n old-project
```

> ***Hint:*** The environment file in the project repository remains; the environment can be recreated later if work resumes.

(3.) Reclaim disk space from the package cache.

```
mamba clean --all
```

> ***This is why:*** This removes downloaded packages no longer used by any environment plus index caches. Existing environments are unaffected because they reference cached files via hardlinks.

Troubleshooting

Per-project environment workflow

In Step 1:

- An environment with both conda and pip installs becomes inconsistent or breaks on an apparently unrelated update.
  - Install conda packages first; install pip packages after, listed under a `pip:` section in `environment.yml`. Never use `pip install` for a package also available on conda-forge.

In Step 2:

- The solver takes minutes or appears stuck after `mamba install` or `mamba env create`.
  - Confirm you are running mamba and not conda. Confirm channel priority is set to `strict`.
  - For very large environments, split heavy stacks (full Bioconductor) into a separate environment from your lighter analysis stack.
- Running `mamba` in a fresh terminal returns "command not found".
  - The shell init step did not run. Execute `~/miniforge3/bin/mamba init` for your shell and restart the terminal. Confirm the appropriate dotfile (`~/.zshrc`, `~/.bashrc`) was modified.
- Environment activation fails in non-interactive shells. A cron job, systemd unit, or build script cannot find `mamba activate`.
  - Source the conda hook explicitly at the top of the script: `source ~/miniforge3/etc/profile.d/conda.sh`, then `mamba activate my-project`.
- Lockfile generation fails with a solver error.
  - The dependency graph is unsatisfiable. Loosen version pins in `environment.yml` or remove the most recently added package and try again to identify the conflict.
- A package looks correctly installed, but the project's scripts cannot import it — or worse, the package shows up in `base` instead of the project environment.
  - The install ran against the wrong environment. The two common variants: (a) no env was activated, so `mamba install <pkg>` defaulted to `base`; (b) a different project's env was activated, and the install landed there. Diagnose with `echo $CONDA_DEFAULT_ENV` (current env), `which python` / `which R` (which interpreter the shell will run), and `mamba list -n base | grep <pkg>` (whether the package leaked into base).
  - Rebuild both affected envs from their committed specs (see "Recovering a broken environment"). The wrong env then no longer carries the stray package, the right env has whatever you previously committed, and you re-run the intended install once before re-exporting and re-locking. If `base` is the wrong env, rebuilding is not an option (do not rebuild `base`) — instead remove the stray package directly: `mamba remove -n base <pkg>`. The per-project workflow's rule that `base` stays empty makes this remediation unambiguous.
  - Prevent recurrence by passing `-n <env-name>` explicitly on every `mamba install` rather than relying on activation state. Verify the shell prompt shows the active env name (mamba init configures this by default in `~/.zshrc` / `~/.bashrc`); if it does not, the activation hook did not run and any "active" env is fiction.

- Installation aborts partway through linking with `Cannot find a valid extracted directory cache for '<package>.conda' / Package cache error`, or otherwise leaves the environment in an inconsistent state.
  - Rebuild the env from the committed lockfile (see "Recovering a broken environment"). The in-place repairs below are for cases where rebuilding is genuinely impossible — no lockfile committed, or restoration costs measured in hours.
  - If you must repair in place: clear the cache and re-run the same install command. Mamba is idempotent and will finish what was started: `mamba clean --packages --tarballs`, then re-run the failed `mamba install ...`.
  - If a specific package is missing from the env and `--force-reinstall <package>` reports "Nothing to do", the package is no longer in mamba's history. Force-reinstall the top-level parent that pulled it in instead (find it with `mamba repoquery whoneeds <package> -n <env>`), which re-resolves the parent's dependencies and pulls the missing transitive dep back. Do not install transitive dependencies by name as an explicit request, as this pollutes `environment.yml` with packages that are not real project dependencies.
  - If the cache error recurs on a clean cache, check that `~/miniforge3/` lives on a normal local filesystem. Conda's package cache cannot live on iCloud Drive, OneDrive, Dropbox, or any path under `~/Library/Mobile Documents/`. Reinstall miniforge under a local path (the default `~/miniforge3`) if it has drifted into a synced folder.
- A CRAN or Bioconductor package has no `r-` or `bioconductor-` equivalent.
  - Install it from inside R using `install.packages()` or `BiocManager::install()`. Note in the project README that this package is not captured by `environment.yml`. If the package is critical, consider switching the project to native R with `renv`.
- An R script calls `install.packages()` or `BiocManager::install()` directly, either in your own code or in an external script you are running.
  - Calling `install.packages()` inside a script is widely considered poor practice in the R community since it silently modifies the recipient's library without consent (see Bryan, "Project-oriented workflow", tidyverse.org, 2017). Scripts should only call `library()` to declare what they need, not installing it.
  - For packages in your own scripts: remove the call and add the package to `environment.yml` instead; re-export and re-lock.
  - For external scripts you cannot modify: the call will install into R's user library outside the conda environment and will not travel with the project. Note the dependency in the README and add a conda equivalent to `environment.yml` where one exists.

Sharing and managing environment files

In Step 1:

- An environment recreated on an Apple Silicon Mac fails because some packages have no `osx-arm64` build.
  - Force the architecture for the affected environment: `CONDA_SUBDIR=osx-64 mamba env create -n my-project -f environment.yml`. The environment will run under Rosetta 2; use as a fallback only.

Trial upgrade of a pinned dependency

In Step 3:

- `conda-lock lock` fails with a conflict involving `python`, `numpy`, `r-base`, or other transitively-pinned packages.
  - The pin set is jointly infeasible against the new target version. The classic case: `numpy=1.26`, `pandas=2.2`, `pyarrow=16`, `pytorch=2.4`, `transformers=4.44` all pinned tightly alongside `python=3.14` — none of those versions have `cp314` wheels yet, and some have hard constraints against newer NumPy ABIs. Loosen the pins on the packages that block the upgrade. Keep only pins that the analysis genuinely depends on (e.g. `numpy<2` if the project relies on a torch ABI that requires it) and let the solver pick everything else.
- `mamba search` reports an outdated newest version (e.g. `r-base` tops out at 4.3 although 4.5 is on conda-forge).
  - The local package index cache has gone stale. Refresh and re-search: `mamba clean --index-cache; mamba search r-base -c conda-forge`.
  - If the freshest version still looks too old, check architecture: `mamba search r-base -c conda-forge --platform osx-arm64` (or `osx-64`, `linux-64`). Some packages lag on Apple Silicon; the `CONDA_SUBDIR=osx-64` fallback applies (see "Sharing and managing environment files" troubleshooting).

Resources

Per-project environment workflow

```yaml
name: my-project
channels:
  - conda-forge
  - bioconda
  - defaults
dependencies:
  - python=3.14
  - numpy=2.3
  - pandas=2.3
  - scipy
  - scikit-learn
  - matplotlib
  - seaborn
  - jupyterlab
  - r-base=4.5
  - r-essentials
  - r-tidyverse
  - bioconductor-deseq2
  - samtools
  - bwa
```

In Step 1: An environment file with some commonly used packages. Adjust dependencies based on your project's needs.

Purpose	Language	Package	Channel	Notes
Runtime	Python	`python=3.14`	conda-forge	Pin minor version
Runtime	R	`r-base=4.5`	conda-forge	Pin minor version
Runtime	R	`r-essentials`	conda-forge	knitr, rmarkdown, and common R extensions
Notebooks	Python	`jupyterlab`	conda-forge	Interactive notebook interface
Notebooks	R	`r-irkernel`	conda-forge	R kernel for Jupyter; run `R -e "IRkernel::installspec()"` after install
Data analysis	Python	`numpy=2.3`	conda-forge	Arrays and linear algebra; pin minor version
Data analysis	Python	`pandas=2.3`	conda-forge	Tabular data (DataFrames); pin minor version
Data analysis	Python	`scipy`	conda-forge	Statistics, signal processing, optimization
Data analysis	Python	`scikit-learn`	conda-forge	Machine learning models and preprocessing
Data analysis	R	`r-tidyverse`	conda-forge	ggplot2, dplyr, tidyr, purrr, readr, etc.
Visualization	Python	`matplotlib`	conda-forge	General-purpose plotting
Visualization	Python	`seaborn`	conda-forge	Statistical visualization built on matplotlib
Differential expression	R	`bioconductor-deseq2`	bioconda	RNA-seq count data
Differential expression	R	`bioconductor-edger`	bioconda	RNA-seq count data
Sequence alignment	—	`bwa`	bioconda	Short-read alignment (BWA-MEM)
Sequence processing	—	`samtools`	bioconda	SAM/BAM file manipulation and statistics
Sequence processing	—	`bcftools`	bioconda	VCF/BCF variant calling and processing
Quality control	—	`multiqc`	bioconda	Aggregated QC reports across pipeline tools

In Step 1: Common dependencies in scientific computing.

Change log

2026-05-10	Benjamin C. Buchmuller	Initial commit; scope informed by Robert Haase, "Managing Scientific Python environments using Conda, Mamba and friends", FocalPlane (Company of Biologists, 8 December 2022).

From the *Lab Protocols* collection by B. C. Buchmuller and contributors (2026). Licensed under Creative Commons Attribution Share Alike 4.0 International; see https://creativecommons.org/licenses/by-sa/4.0/ for the terms.

For research use. Provided in good faith, without warranty or liability for any use or results. Users are responsible for compliance with local regulations and institutional policies.

Current when printed. Visit https://benjbuch.github.io/check/ or scan the QR code to check for updates.

4166ca8